A D PATEL INSTITUTE OF TECHNOLOGY
Department of Information Technology

A MINI PROJECT REPORT ON

CRICKET TOURNAMENT SCHEDULER USING GREEDY ALGORITHMS

(Greedy Graph Colouring and Greedy Match Scheduling for a Round-Robin Cricket Tournament, Implemented in Python)

Subject: Design and Analysis of Algorithm

Submitted by

Dhruv Ka. Patel
Enrollment No.: 12402080601032
Department: IT-A | Semester V

Under the guidance of

Minakshi Patil

Information Technology
A D Patel Institute of Technology

Academic Year 2026 – 27

Certificate

This is to certify that the mini project report entitled **"Cricket Tournament Scheduler Using Greedy Algorithms"** is a bonafide record of the work carried out by **Dhruv Ka. Patel** (Enrollment No. 12402080601032), a student of Semester V, Department of Information Technology (IT-A), A D Patel Institute of Technology, CVM University, Vallabh Vidyanagar, for the subject **Design and Analysis of Algorithm** during the academic year 2026 – 27.

The work has been completed under my guidance and supervision. To the best of my knowledge, the report has not been submitted to any other institute or university for the award of any degree, diploma or certificate.

Minakshi Patil
Project Guide
Department of Information Technology

Head of Department
Department of Information Technology
A D Patel Institute of Technology

Date: _____

Place: Vallabh Vidyanagar

Acknowledgement

I would like to express my sincere gratitude to everyone who helped me in completing this mini project on **Cricket Tournament Scheduler Using Greedy Algorithms**.

First of all, I thank my guide, **Minakshi Patil**, for her valuable guidance, constant encouragement and useful suggestions during every stage of the project. Her feedback on the choice of algorithm, the analysis and the structure of the report helped me to improve the quality of my work.

I am also thankful to the Head of the Department of Information Technology and to all the faculty members of A D Patel Institute of Technology for providing the laboratory facilities and the learning environment in which this project could be carried out.

I thank the authors of the textbooks and papers listed in the references, which I used to understand the greedy method, graph colouring and sports scheduling. Finally, I thank my classmates, friends and family for their support and motivation.

Dhruv Ka. Patel
Enrollment No.: 12402080601032

Abstract

A cricket tournament has to answer three questions: who plays whom, on which day, and at which ground. In a single round-robin every team plays every other team once, but a team cannot play two matches on the same day, it needs rest days between its matches, and each ground can host only one match per day. Choosing a timetable that respects these rules and finishes the tournament early is a scheduling problem, and the greedy method is a natural way to solve it quickly.

This project applies two greedy algorithms to a round-robin tournament. First, the matches are the vertices of a conflict graph (two matches conflict when they share a team) and are coloured with the Welsh–Powell greedy algorithm; each colour is a round in which no team plays twice. For 8 teams it gives 7 rounds, which is optimal because the 7 matches of one team form a clique; for other sizes the colouring is wasteful (15 rounds against a lower bound of 9 for 10 teams). Second, a greedy timetable builder fills each day by repeatedly choosing, among the matches that are feasible, the one whose two teams have the most matches left, subject to the number of grounds and the rest-day rule.

The scheduler was implemented in Python and compared with first-fit schedulers (fixture-list order, and random order averaged over 20 seeds) in four set-ups. The greedy timetable finished the tournament in 15 days against 18 in the Small set-up, 16 against 18 in Standard, 22 against 27 in Large, and 19 against 19 in Congested, where it only tied. In two set-ups it reached the proven lower bound. The report gives the algorithms, their complexity, the code and the test results, and it states clearly that the greedy scheduler is a heuristic whose results hold for the model used.

Keywords: greedy algorithm, graph colouring, Welsh–Powell, tournament scheduling, round-robin, cricket, time complexity.

Table of Contents

1. Introduction **7**

2. Review of Existing Approaches **8**

3. Problem Statement and Objectives **8**

4. Scope and System Requirements **8**

5. Algorithmic Techniques Used **9**

6. Algorithm Design and Analysis **10**

7. Implementation **13**

8. Testing and Results **17**

9. Advantages and Limitations **21**

10. Future Scope **21**

11. Conclusion **21**

References **22**

List of Figures

Figure 1: Round-robin: 8 teams and 28 matches....7
Figure 2: Match-conflict graph coloured by the greedy algorithm....11
Figure 3: Flowchart of the greedy timetable builder....12
Figure 4: Days to finish: first-fit schedulers vs greedy scheduler....19
Figure 5: Matches completed day by day (Standard set-up)....19

List of Tables

Table 1: Comparison of scheduling approaches....8
Table 2: Functional requirements....9
Table 3: Hardware and software requirements....9
Table 4: Algorithmic techniques and where they are used....10
Table 5: Size of the match-conflict graph....10
Table 6: Experiment parameters....12
Table 7: Tournament set-ups....12
Table 8: Time and space complexity....13
Table 9: Functions of the program....17
Table 10: Logic test cases and results....18
Table 11: Experiment results....18
Table 12: Effect of the selection rule....20
Table 13: Rounds from greedy colouring vs lower bound....20

1. Introduction

1.1 Overview

Every season thousands of cricket matches have to be fitted into a calendar. A tournament board must decide which teams meet, on which day and at which ground, and a poor timetable makes the tournament longer than necessary or leaves some teams playing in quick succession while others wait for days. This project studies how the greedy method of algorithm design can build a good timetable with a simple and fast program.

The project is developed for the subject Design and Analysis of Algorithm. It is intentionally small, so that the algorithms, their correctness argument and their running time can be analysed completely, yet it is realistic enough to compare the result with the common approach of simply placing matches in the order of the fixture list.

1.2 Cricket Tournament Scheduling in Brief

In a single round-robin tournament with n teams every pair of teams plays exactly once, which gives n(n − 1)/2 matches; 8 teams play 28 matches, shown in Figure 1. The timetable must obey three rules. (1) A team cannot play two matches on the same day. (2) A team needs a number of rest days between two of its matches. (3) A ground can host only one match per day. A **round** is a set of matches in which no team appears twice, so all its matches could be played at the same time. A **timetable** gives every match a day and a ground.

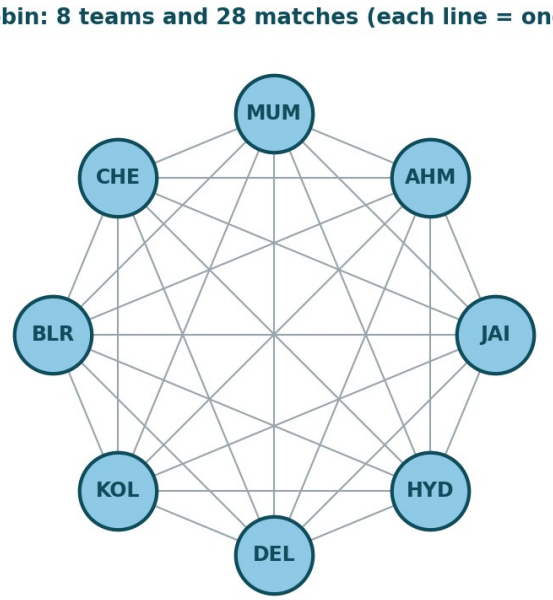


Figure 1: Round-robin: 8 teams and 28 matches

1.3 Why a Greedy Algorithm

Finding the best timetable by trying all possibilities is not practical, because the number of possible timetables grows extremely fast with the number of matches. A greedy algorithm does not try to solve the whole problem in one step. It repeatedly makes the choice that looks best at the moment, here “play the match whose teams still have the most matches left”, and never reconsiders it. This makes the algorithm quick, simple to code and easy to analyse, which is why it suits a student project in algorithm design and analysis.

1.4 Organization of the Report

Chapter 2 reviews existing approaches. Chapter 3 states the problem and objectives. Chapter 4 defines the scope and requirements, and Chapter 5 explains the algorithmic techniques used. Chapter 6 gives the design and the complexity analysis, Chapter 7 the implementation with the complete code, and Chapter 8 the testing and results. Chapters 9 to 11 discuss the advantages and limitations, the future scope and the conclusion.

2. Review of Existing Approaches

Manual fixture lists are written by the organisers, often with the help of a spreadsheet, and checked by hand against the rules. A computer version of this idea is the **first-fit** scheduler, which takes the matches in a fixed order and puts each one on the earliest day where it fits. It is simple, but its result depends strongly on the order of the list. The **circle method** (Berger tables) is a classical constructive way of dividing a round-robin into n − 1 rounds for an even number of teams, but it knows nothing about grounds or rest days. **Optimisation methods** such as integer programming, constraint programming or local search can model all the rules and search for the best timetable, and they are used for real leagues [4][5], but their running time grows quickly with the size of the tournament and they need a solver and a full model.

A greedy scheduler sits between these approaches. Like an optimisation method it respects all the rules by construction, and like a fixture list it is fast. Its weakness is that it decides one match at a time and cannot guarantee the best timetable. Table 1 compares the approaches.

Table 1: Comparison of scheduling approaches

Feature	First-fit (list order)	Greedy (this project)	Optimisation (ILP / CP)
Respects grounds and rest days	Yes, if coded	Yes, built into the rule	Yes, if modelled
Time to build a timetable	Fast	Fast, about $O(M^2)$	Grows quickly with size
Quality of the timetable	Depends on the list order	Good in the tests, not guaranteed	Best, for its model
Simple to implement	Yes	Yes	No
Guarantee of best timetable	No	No (heuristic)	Yes, for its model

3. Problem Statement and Objectives

3.1 Problem Statement

Given n teams, g grounds and a rest rule of r days between the matches of a team, design algorithms that (a) divide the n(n − 1)/2 matches of a single round-robin into rounds in which no team plays twice, using as few rounds as possible, and (b) build a timetable that gives every match a day and a ground, so that no team plays twice on a day, every team has at least r days of rest between its matches, no ground hosts two matches on a day, the tournament finishes in few days and the teams finish at about the same time.

3.2 Objectives

- To model the matches as a conflict graph and colour it with a greedy algorithm to obtain rounds.
- To design a greedy rule that chooses the next match of each day from the matches that are still possible.
- To compute a lower bound on the number of days, so that the quality of a timetable can be judged.
- To analyse the time and space complexity of each algorithm.
- To implement the algorithms in Python and test them on four tournament set-ups.
- To compare the result with first-fit schedulers and to report the limits of the method.

4. Scope and System Requirements

4.1 Scope

The project covers one single round-robin tournament of 6 to 10 teams, played on 1 to 4 identical grounds with one match per ground per day and a rest rule of 1 or 2 days. Home and away matches, double round-robin, play-offs and knock-out stages, travel distances, weather, television slots, umpire availability and real fixture data are outside the scope and are listed under future scope.

4.2 Functional Requirements

Table 2: Functional requirements

ID	Requirement
FR1	The program shall build the list of matches and the conflict graph of the matches.
FR2	The program shall group the matches into conflict-free rounds with a greedy colouring algorithm.
FR3	The program shall build a timetable day by day, choosing greedily from the matches left by the teams and the days they have been idle.
FR4	The program shall respect the number of grounds, the rule that a team plays once a day and the rest-day rule.
FR5	The program shall compute the lower bound on the days, the days used, the longest gap between two matches of a team and the finish spread of the teams.
FR6	The program shall also run first-fit schedulers for comparison and print the results of all of them.

4.3 Non-Functional Requirements

- **Correctness:** every match is played exactly once, no team plays twice on a day, the rest rule is kept and no round contains two matches of the same team.
- **Efficiency:** the timetable of a small tournament must be built in a few milliseconds.
- **Repeatability:** the greedy scheduler is deterministic; the random baseline uses fixed seeds 0 to 19, so the same numbers appear on every run.
- **Readability:** the program is a single Python file of about 220 lines with small functions.

4.4 System Requirements

The program uses only the Python standard library. The requirements used for development and testing are given in Table 3.

Table 3: Hardware and software requirements

Component	Specification
Processor	Any modern processor (Intel Core i3 or equivalent and above)
RAM	4 GB or more
Operating system	Windows, Linux or macOS
Language	Python 3.8 or newer (standard library only: math, random, itertools)
Code editor	Visual Studio Code (or any text editor)

5. Algorithmic Techniques Used

5.1 The Greedy Method

A greedy algorithm builds a solution step by step. At each step it picks, from the candidates available, the one that gives the best immediate gain according to a *selection function*, checks that the choice is *feasible*, and never undoes it. A greedy algorithm is guaranteed to be optimal only when the problem has the *greedy-choice property* (a locally best choice can be extended to a globally best solution) and *optimal substructure*. Activity selection, Huffman coding and Dijkstra's shortest path algorithm are classic examples [1]. When these properties cannot be proved, a greedy algorithm is used as a fast heuristic, and its quality is judged by analysis and experiment.

5.2 Greedy Graph Colouring

Colouring a graph means giving each vertex a colour so that adjacent vertices have different colours. Finding the minimum number of colours is NP-hard in general. The **Welsh–Powell** algorithm [3] is a greedy heuristic: sort the vertices by decreasing degree, then give each vertex the smallest colour not used by its neighbours. It never needs more than $\Delta + 1$ colours, where Δ is the largest degree. In this project each vertex is a match, each edge joins two matches that share a team, and each colour is a round.

5.3 Greedy Match Selection

Match selection is the greedy choice repeated over the days. The candidate set is the set of matches that are feasible today (both teams are unused today and rested, and a ground is free). The selection function is the number of matches left for the two teams plus an aging bonus of 0.25 for every day the two teams have been idle. The match with the largest score is placed on the next free ground, and the step is repeated until the grounds of the day are full or no match is feasible.

5.4 Why the Greedy Colouring Never Fails

When a vertex v is coloured, at most deg(v) of its neighbours have a colour already, so at most deg(v) colours are forbidden. Among the colours 0, 1, …, deg(v) at least one is free, so the algorithm always finds a colour not greater than deg(v) and uses at most $\Delta + 1$ colours in total. In the conflict graph of n teams $\Delta = 2(n - 2)$, so the bound is $2n - 3$ rounds, which is 13 for 8 teams. The actual result for 8 teams is 7, much better than the worst case.

5.5 Techniques and Their Use

Table 4: Algorithmic techniques and where they are used

Technique	Used for	Function in code
Graph modelling	Matches as vertices, shared teams as edges	build_conflict_graph()
Greedy colouring (Welsh–Powell)	Grouping matches into rounds	greedy_rounds()
Greedy selection with aging	Choosing the next match of each day	greedy_timetable()
First-fit baseline	Comparison with a simple scheduler	first_fit()
Lower bound and measures	Judging the quality of a timetable	lower_bound(), measures()
Asymptotic analysis	Comparing running time and space	Chapter 6

6. Algorithm Design and Analysis

6.1 Conflict Graph

The vertices are the $n(n - 1)/2$ matches. Two matches are joined by an edge if they share a team, because a team cannot play both on the same day. The $n - 1$ matches of one team all share that team, so they form a **clique** of size $n - 1$. Two different matches can share at most one team, so the number of edges is $E = n \cdot C(n - 1, 2)$ and every match has degree $2(n - 2)$. Table 5 shows the size of the graph for several tournaments. For 8 teams the graph has 28 vertices and 168 edges.

Table 5: Size of the match-conflict graph

Teams n	Matches M	Conflicts E	Degree 2(n − 2)	Bound Δ + 1 = 2n − 3
4	6	12	4	5
6	15	60	8	9
8	28	168	12	13
10	45	360	16	17
12	66	660	20	21

6.2 Greedy Colouring Algorithm

```
WELSH-POWELL(G)
 1 order <- vertices of G sorted by decreasing degree (ties: lower match number first)
 2 for each vertex v in order
 3     used <- { colour[u] : u is a neighbour of v and u is coloured }
 4     colour[v] <- smallest non-negative integer not in used
 5 return the colour classes (each class is one round)
```

Because every match has the same degree, the order is simply the fixture list. For 8 teams the algorithm gives 7 rounds. **Optimality for 8 teams:** the 7 matches of one team conflict with each other, so they form a clique of 7 vertices and any colouring needs at least 7 colours; the greedy result uses exactly 7, so for 8 teams the greedy colouring is optimal. Figure 2 shows the coloured graph for 4 teams, where 3 rounds are obtained.

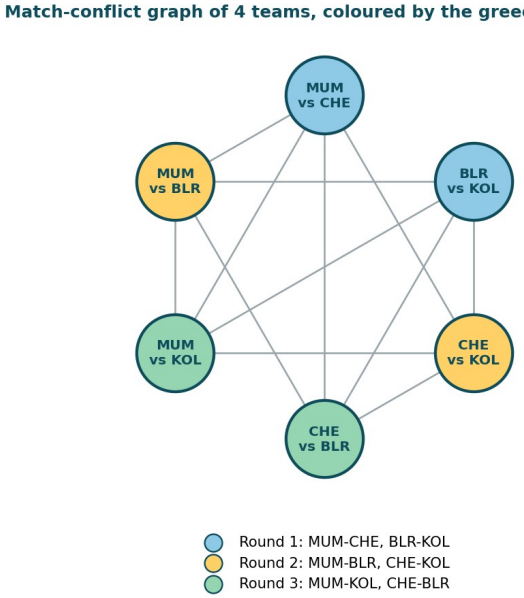


Figure 2: Match-conflict graph coloured by the greedy algorithm

Limit of the method. For other team numbers the greedy colouring needs more rounds than necessary. With the teams numbered from 0, the program showed that the colour of the match between teams i and j is (i XOR j) − 1 for every tournament from 2 to 20 teams, so the number of rounds is the smallest power of two that is not below n, minus 1. This is optimal when n is a power of two (4, 8, 16) but wasteful otherwise: 7 rounds for 6 teams (lower bound 5) and 15 for 10 teams (lower bound 9). The results are in Table 13. If only the rounds are needed, the circle method gives n − 1 rounds for any even n.

6.3 Greedy Timetable Builder

```
GREEDY-TIMETABLE(matches, grounds, rest)
 1 day <- 0;  pending <- all matches;  left[t] <- number of matches of team t
 2 while pending is not empty
 3     day <- day + 1;  used <- { }
 4     repeat at most "grounds" times
 5         F <- matches (a, b) in pending with a, b not in used and both rested (day - last >
rest)
 6         if F is empty: stop for today
 7         m <- the match in F with the largest  left[a] + left[b] + 0.25 * (idle[a] + idle[b])
 8         place m on the next free ground;  add a and b to used;  last[a], last[b] <- day
 9         left[a] <- left[a] - 1;  left[b] <- left[b] - 1;  remove m from pending
10 return the timetable
```

Here idle[t] = day − last[t], the number of days since team t last played, and ties are broken in favour of the lower match number. **Why greedy:** a team with k matches left needs at least (k − 1)(r + 1) + 1 more days, so the teams with the most matches left decide when the tournament can end. Playing their matches first keeps them from becoming the bottleneck at the end. This single-step argument justifies the choice, but it does not prove that the sequence of choices is optimal, because a match placed today can block a better choice later. The builder is therefore a heuristic and is evaluated by experiment.

6.4 Flowchart

Figure 3 shows the complete working of the timetable builder. The matches are listed once; after that the builder repeats the choose–place–update cycle until every match has a day.

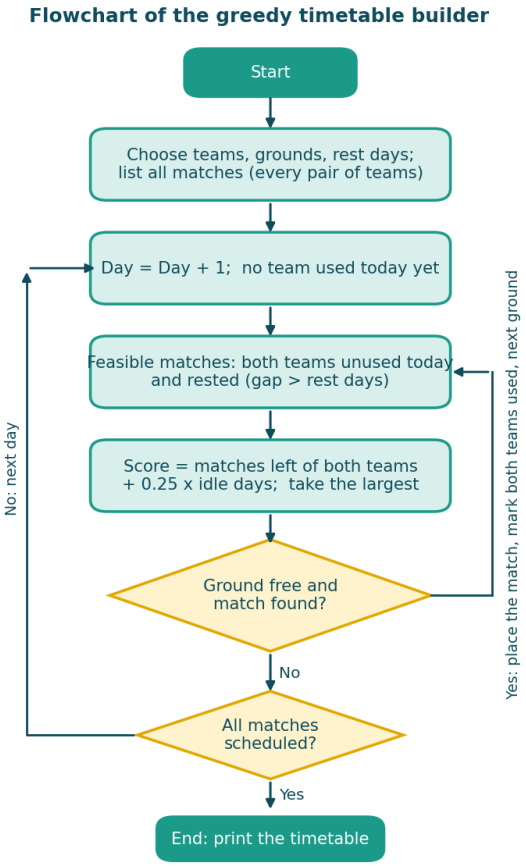


Figure 3: Flowchart of the greedy timetable builder

6.5 Experiment Model

Table 6: Experiment parameters

Parameter	Value	Meaning
Teams	6 to 10	Single round-robin, every pair of teams plays once
Grounds	1 to 4	Each ground hosts one match per day
Rest days	1 or 2	Days without a match between two matches of a team
Aging factor	0.25	Bonus per idle day of a team in the selection score
Tie-break	Lowest match number	Makes the greedy scheduler deterministic
Random baseline	20 seeds (0–19)	Each seed shuffles the fixture order for the first-fit scheduler

Four set-ups are used (Table 7). The *lower bound* is the larger of two limits. The grounds limit is $\lceil M / g \rceil$ days, because at most g matches are played per day. The team limit is $(n - 1)(r + 1) - r$ days, because a team has $n - 1$ matches that are at least $r + 1$ days apart. No timetable can be shorter than this bound, so a timetable that reaches it is optimal.

Table 7: Tournament set-ups

Set-up	Teams	Matches	Grounds	Rest days	Lower bound (days)
Small	6	15	1	1	15
Standard	8	28	2	1	14
Large	10	45	3	1	17
Congested	8	28	4	2	19

Three measures are reported. **Days** is the number of days from the first to the last match. **Longest gap** is the largest number of days between two consecutive matches of one team; a large value means a team waits for a

long time. **Finish spread** is the difference between the day on which the last team finishes and the day on which the first team finishes; a small value means that all teams finish at about the same time. The two baselines are first-fit with the fixture-list order and first-fit with a random order (mean of 20 seeds).

6.6 Complexity Analysis

Let n be the number of teams, $M = n(n-1)/2$ the number of matches, E the number of conflicts, g the number of grounds and D the number of days of the timetable. The results are summarised in Table 8.

Table 8: Time and space complexity

Step	Time	Space
Build conflict graph	$O(M + E) = O(n^3)$	$O(M + E)$
Greedy colouring (sort + neighbour scan)	$O(M \log M + M \cdot \Delta) = O(n^3)$	$O(M)$
One greedy pick (scan of pending matches)	$O(M)$	$O(M)$
Whole greedy timetable	$O(D \cdot g \cdot M)$	$O(M)$
First-fit baseline	$O(M \cdot D \cdot n)$	$O(M)$

The number of days D is at least M / g, and it is never more than $(r + 1) \cdot M$, because after r + 1 days without a match every team is rested and a match can be placed again. As $D \cdot g$ is close to M in the experiments, the running time of the greedy timetable is about $O(M^2) = O(n^4)$. On the development computer the timetable was built in about 0.1 ms for 8 teams, 0.6 ms for 12 teams, 1.3 ms for 16 teams and 3.3 ms for 20 teams (with n/4 grounds); these timings depend on the computer. A tournament of this size is therefore scheduled instantly.

6.7 Worked Example of One Decision

Take the Small set-up (6 teams, 1 ground, 1 rest day). Day 1 has MUM vs CHE and day 2 has BLR vs KOL. On day 3 the teams BLR and KOL played yesterday, so they must rest. MUM and CHE played on day 1 and DEL and HYD have not played yet, so five matches are feasible. The score of a match is the matches left of its two teams plus 0.25 times their idle days (a team that has not played yet counts as idle since day −1):

```
Match         left   idle   score
DEL vs HYD     5+5    4+4   10 + 0.25 x 8 = 12.00   <- largest score, chosen
MUM vs DEL     4+5    2+4    9 + 0.25 x 6 = 10.50
MUM vs HYD     4+5    2+4    9 + 0.25 x 6 = 10.50
CHE vs DEL     4+5    2+4    9 + 0.25 x 6 = 10.50
CHE vs HYD     4+5    2+4    9 + 0.25 x 6 = 10.50
```

A first-fit scheduler in fixture-list order would play MUM vs DEL, the first feasible match in the list. The greedy rule chooses DEL vs HYD, because both teams have not played and have the most matches left, so they are the bottleneck of the tournament. After the decision both teams have 4 matches left and last played on day 3. This example was checked by running the program, which also places DEL vs HYD on day 3.

6.8 Greedy Method Compared with Other Paradigms

A **brute-force** search would try every possible assignment of days and grounds to the matches, and the number of assignments grows exponentially with the number of matches. **Dynamic programming** needs a state that records which matches are finished and when each team last played; the subsets of the 28 matches of an 8-team tournament alone number $2^{28} = 268\,435\,456$, so the table becomes too large very quickly. The **greedy method** needs neither: it looks only at the matches that are possible today, so it runs in a few milliseconds. The price is that it gives no guarantee of the best timetable, which is why the project measures its result against a lower bound and against other schedulers instead of claiming optimality.

7. Implementation

The whole project is the single file `cricket_tournament_scheduler.py`. It has the main functions in Table 9 and a `main()` function that runs the self-check and prints the results. The listing is given below.

```
"""Cricket Tournament Scheduler Using Greedy Algorithms (DAA mini project).
Part 1: model the matches of a round-robin tournament as a conflict graph (two
```

```
        matches conflict if they share a team) and colour it greedily
        (Welsh-Powell). Each colour is one "round" in which no team plays twice.
Part 2: build a day-by-day timetable. Every day the greedy rule picks the most
        urgent match that fits the venues and the rest-day rule.
Part 3: compare it with first-fit schedulers on four tournament set-ups.
"""
import math
import random
from itertools import combinations

TEAMS = ["MUM", "CHE", "BLR", "KOL", "DEL", "HYD", "JAI", "AHM", "PUN", "LKO", "GUW", "RAJ"]
GROUNDS = ["Narendra Modi Stadium", "Wankhede Stadium", "Eden Gardens", "Chepauk Stadium"]
AGING = 0.25      # bonus per day a team has been idle (keeps every team playing regularly)

SCENARIOS = {     # teams, grounds (matches per day), rest days needed between a team's matches
    "Small":     dict(teams=6, venues=1, rest=1),
    "Standard":  dict(teams=8, venues=2, rest=1),
    "Large":     dict(teams=10, venues=3, rest=1),
    "Congested": dict(teams=8, venues=4, rest=2),
}


# ---------------------------------------------------------------- Part 1
def make_matches(n):
    """Single round-robin: every pair of teams plays once -> n(n-1)/2 matches."""
    return list(combinations(range(n), 2))


def build_conflict_graph(matches):
    """Vertex = match. Edge = the two matches share a team (cannot be played together)."""
    graph = {i: set() for i in range(len(matches))}
    by_team = {}
    for i, (a, b) in enumerate(matches):
        by_team.setdefault(a, []).append(i)
        by_team.setdefault(b, []).append(i)
    for group in by_team.values():            # the matches of one team form a clique
        for i, j in combinations(group, 2):
            graph[i].add(j)
            graph[j].add(i)
    return graph


def greedy_rounds(graph):
    """Welsh-Powell greedy colouring: highest degree first, smallest free colour."""
    order = sorted(graph, key=lambda v: (-len(graph[v]), v))
    colour = {}
    for v in order:
        used = {colour[u] for u in graph[v] if u in colour}
        c = 0
        while c in used:
            c += 1
        colour[v] = c
    rounds = [[] for _ in range(max(colour.values()) + 1)]
    for v in sorted(graph):
        rounds[colour[v]].append(v)
    return rounds


def circle_method(n):
    """Classical constructive round-robin (n even): n-1 rounds. Used only as a reference."""
    teams, rounds = list(range(n)), []
    for _ in range(n - 1):
        rounds.append([tuple(sorted((teams[i], teams[n - 1 - i]))) for i in range(n // 2)])
        teams = [teams[0], teams[-1]] + teams[1:-1]
    return rounds


# ---------------------------------------------------------------- Part 2
def greedy_timetable(matches, n, venues, rest, rule="priority"):
    """Day by day, repeatedly pick the best match that is feasible today.
```

```
    rule: 'list' = first feasible match in fixture list (no priority),
          'remaining' = teams with most matches left first,
          'priority' = remaining matches + AGING * idle days (used)."""
    remaining = [sum(t in m for m in matches) for t in range(n)]
    last = [-rest] * n                      # day of the team's last match
    pending = list(range(len(matches)))
    schedule, day = [], 0
    while pending:
        day += 1
        used = set()
        for venue in range(venues):
            best, best_score = None, None
            for m in pending:
                a, b = matches[m]
                if a in used or b in used or day - last[a] <= rest or day - last[b] <= rest:
                    continue
                if rule == "list":
                    score = -m
                else:
                    score = remaining[a] + remaining[b]
                    if rule == "priority":
                        score += AGING * ((day - last[a]) + (day - last[b]))
                if best_score is None or score > best_score:
                    best, best_score = m, score
            if best is None:
                break
            a, b = matches[best]
            schedule.append((day, venue, best))
            used |= {a, b}
            last[a] = last[b] = day
            remaining[a] -= 1
            remaining[b] -= 1
            pending.remove(best)
    return schedule


def first_fit(matches, order, n, venues, rest):
    """Baseline: take matches in the given order, put each on the earliest day that fits."""
    team_days = [set() for _ in range(n)]
    load, schedule = {}, []
    for m in order:
        a, b = matches[m]
        d = 1
        while load.get(d, 0) >= venues or any(abs(d - x) <= rest for x in team_days[a] | team_days[b]):
            d += 1
        schedule.append((d, load.get(d, 0), m))
        load[d] = load.get(d, 0) + 1
        team_days[a].add(d)
        team_days[b].add(d)
    return sorted(schedule)


# ---------------------------------------------------------------- Measures and checks
def lower_bound(n, venues, rest):
    """No schedule can finish earlier than this many days."""
    per_day = math.ceil(n * (n - 1) / 2 / venues)          # grounds limit
    per_team = (n - 1) * (rest + 1) - rest                  # rest rule limit
    return max(per_day, per_team)


def measures(schedule, matches, n):
    days = max(d for d, _, _ in schedule)
    played = [[] for _ in range(n)]
    for d, _, m in schedule:
        for t in matches[m]:
            played[t].append(d)
    gaps = [y - x for p in played for x, y in zip(sorted(p), sorted(p)[1:])]
    finish = [max(p) for p in played]
    return {"days": days, "max_gap": max(gaps), "spread": max(finish) - min(finish)}
```

```
def is_valid(schedule, matches, n, venues, rest):
    if sorted(m for _, _, m in schedule) != list(range(len(matches))):
        return False                                        # every match exactly once
    slots = [(d, v) for d, v, _ in schedule]
    if len(set(slots)) != len(slots) or any(v >= venues for _, v in slots):
        return False                                        # one match per ground per day
    played = [[] for _ in range(n)]
    for d, _, m in schedule:
        for t in matches[m]:
            played[t].append(d)
    return all(y - x > rest for p in played for x, y in zip(sorted(p), sorted(p)[1:]))


def self_check():
    for n in (4, 6, 8, 10):
        matches = make_matches(n)
        graph = build_conflict_graph(matches)
        assert all(i in graph[j] for i in graph for j in graph[i])              # symmetric
        rounds = greedy_rounds(graph)
        assert sorted(m for r in rounds for m in r) == list(range(len(matches)))  # each match once
        assert all(not (graph[a] & set(r)) for r in rounds for a in r)            # conflict-free
        assert len(rounds) >= n - 1                                               # lower bound
        assert len(circle_method(n)) == n - 1
    for cfg in SCENARIOS.values():
        n, v, r = cfg["teams"], cfg["venues"], cfg["rest"]
        matches = make_matches(n)
        for rule in ("list", "remaining", "priority"):
            assert is_valid(greedy_timetable(matches, n, v, r, rule), matches, n, v, r)
        assert is_valid(first_fit(matches, list(range(len(matches))), n, v, r), matches, n, v, r)
        assert measures(greedy_timetable(matches, n, v, r), matches, n)["days"] >= lower_bound(n, v, r)
    return True


# ---------------------------------------------------------------- Output
def show_timetable(schedule, matches, venues):
    for day in sorted({d for d, _, _ in schedule}):
        games = [f"{GROUNDS[v]}: {TEAMS[matches[m][0]]} vs {TEAMS[matches[m][1]]}"
                 for d, v, m in schedule if d == day]
        print(f"  Day {day:2d} | " + " | ".join(games))


def main():
    print("Self-check of all algorithms:", "PASSED" if self_check() else "FAILED")

    print("\nPART 1 - greedy colouring of the match-conflict graph")
    print("teams matches edges greedy_rounds lower_bound circle_method")
    for n in (4, 6, 8, 10, 12):
        matches = make_matches(n)
        graph = build_conflict_graph(matches)
        edges = sum(len(s) for s in graph.values()) // 2
        print(f"{n:5d} {len(matches):7d} {edges:5d} {len(greedy_rounds(graph)):13d} "
              f"{n - 1:11d} {len(circle_method(n)):13d}")

    cfg = SCENARIOS["Standard"]
    matches = make_matches(cfg["teams"])
    print("\nPART 2 - greedy timetable, Standard set-up (8 teams, 2 grounds, 1 rest day)")
    show_timetable(greedy_timetable(matches, cfg["teams"], cfg["venues"], cfg["rest"]),
                   matches, cfg["venues"])

    print("\nPART 3 - days needed to finish the tournament (random order = mean of 20 seeds)")
    print(f"{'Scenario':10s} {'LowerBnd':>8s} {'FixtureFF':>9s} {'RandomFF':>9s} {'Greedy':>7s}"
          f" {'maxgap(F/G)':>12s} {'spread(F/G)':>12s}")
    for name, c in SCENARIOS.items():
        n, v, r = c["teams"], c["venues"], c["rest"]
        matches = make_matches(n)
        fix = first_fit(matches, list(range(len(matches))), n, v, r)
        gre = greedy_timetable(matches, n, v, r)
        rnd = []
        for seed in range(20):
```

```
            order = list(range(len(matches)))
            random.Random(seed).shuffle(order)
            rnd.append(measures(first_fit(matches, order, n, v, r), matches, n)["days"])
        mf, mg = measures(fix, matches, n), measures(gre, matches, n)
        print(f"{name:10s} {lower_bound(n, v, r):8d} {mf['days']:9d} {sum(rnd) / 20:9.1f} {mg['days']:7d}"
              f" {mf['max_gap']:>6d}/{mg['max_gap']:<5d} {mf['spread']:>6d}/{mg['spread']:<5d}")


if __name__ == "__main__":
    main()
```

Table 9: Functions of the program

Function	Role
make_matches(n)	Lists the n(n − 1)/2 matches of a single round-robin
build_conflict_graph(matches)	Creates the adjacency sets: two matches are adjacent if they share a team
greedy_rounds(graph)	Welsh–Powell colouring; returns the list of rounds
circle_method(n)	Classical n − 1 rounds, used only as a reference
greedy_timetable(...)	Builds the day-by-day timetable with the greedy rule
first_fit(...)	Baseline: earliest feasible day for each match in a given order
lower_bound(...) and measures(...)	Lower bound on the days; days, longest gap and finish spread
is_valid(...) and self_check()	Check every rule of the timetable and of the rounds
show_timetable(...) and main()	Print the timetable and run all experiments

7.1 Explanation of the Code

- **Conflict graph:** the matches of one team form a clique, so the graph is built by joining every pair of matches that belong to the same team; the adjacency sets are symmetric.
- **Colouring:** the matches are sorted by degree, and for each one the smallest colour not used by an already coloured neighbour is taken.
- **Greedy timetable:** for every day and every ground the loop scans the pending matches, skips those whose teams are used today or not rested, and keeps the match with the largest score; the first best match wins a tie.
- **First-fit:** each match, in the given order, gets the earliest day on which a ground is free and both teams are at least rest + 1 days away from all their other matches.
- **Checks:** `is_valid()` verifies that every match appears once, that no ground is used twice on a day and that every gap between matches of a team is larger than the rest rule; `self_check()` runs it on all set-ups.

7.2 How to Run

1. Save the code as `cricket_tournament_scheduler.py`.
2. Run `python cricket_tournament_scheduler.py` in a terminal.
3. The program prints the self-check, the rounds of the greedy colouring, the timetable of the Standard set-up and the comparison of the schedulers.

8. Testing and Results

8.1 Logic Tests

The correctness of the graph, the colouring and the schedulers was checked with the following tests, all of which were run on the program given above.

Table 10: Logic test cases and results

No.	Test	Observed output	Result
1	Conflict graph is symmetric (4, 6, 8, 10 teams)	If A conflicts with B then B conflicts with A	Pass
2	Size of the graph (4 to 12 teams)	$E = n \cdot C(n-1, 2)$ and every degree $= 2(n-2)$; for 8 teams 168 and 12	Pass
3	Every round is conflict-free (4 to 12 teams)	No two matches of a round share a team	Pass
4	Every match is in exactly one round	28 matches covered once for 8 teams	Pass
5	Lower bound for rounds	The 7 matches of team 0 form a clique of size 7; greedy used 7 rounds	Pass
6	Circle method (reference), 4 to 12 teams	$n-1$ rounds, every pair once, every team once per round	Pass
7	Timetable is valid (4 set-ups, 3 greedy rules, and 20 random first-fit orders each)	Every match once; at most one match per ground per day; no team twice a day; rest rule kept	Pass
8	Days never below the lower bound (4 set-ups)	Greedy days ≥ lower bound in all set-ups	Pass
9	Same input gives same timetable	Two runs of the greedy scheduler gave identical timetables	Pass
10	Smallest tournament (2 teams)	One match on day 1	Pass
11	Wrong timetable is detected	Two matches of one team on the same day rejected by is_valid()	Pass

8.2 Experiment Results

Table 11 gives the results for the four set-ups. The greedy scheduler is deterministic, the fixture-order first-fit is deterministic, and the random-order first-fit is the mean of 20 random orders (seeds 0 to 19).

Table 11: Experiment results

Set-up	Scheduler	Days	Longest gap	Finish spread
Small (lower bound 15)	First-fit, fixture order	18	6	9
	First-fit, random order (mean)	16.6	6.5	5.7
	Greedy scheduler	15	5	3
Standard (lower bound 14)	First-fit, fixture order	18	4	5
	First-fit, random order (mean)	20.6	6.5	4.8
	Greedy scheduler	16	4	1
Large (lower bound 17)	First-fit, fixture order	27	7	10
	First-fit, random order (mean)	26.1	7.5	6.9
	Greedy scheduler	22	3	2
Congested (lower bound 19)	First-fit, fixture order	19	3	0
	First-fit, random order (mean)	22.6	6.2	3.5
	Greedy scheduler	19	3	0

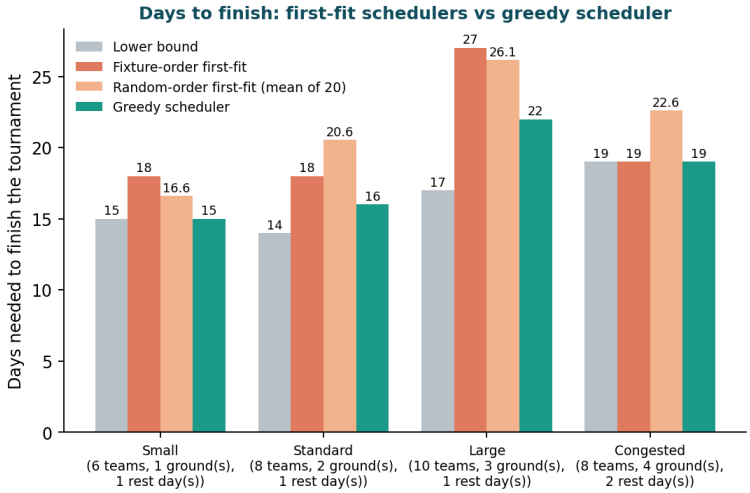


Figure 4: Days to finish: first-fit schedulers vs greedy scheduler

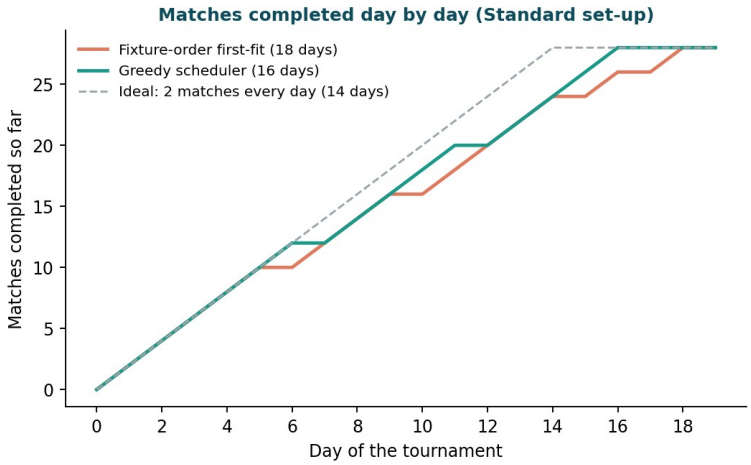


Figure 5: Matches completed day by day (Standard set-up)

8.3 Observations

- Against the fixture-order first-fit the greedy scheduler needs fewer days in three set-ups: 15 against 18 days in Small (17% fewer), 16 against 18 in Standard (11%) and 22 against 27 in Large (19%). In Congested both need 19 days.
- Against the random-order first-fit (mean 16.6, 20.6, 26.1 and 22.6 days) the greedy scheduler is lower in all four set-ups (10%, 22%, 16% and 16% fewer days). It is also never worse than the best of the 20 random orders (15, 19, 24 and 19 days).
- In Small and Congested the greedy timetable reaches the lower bound (15 and 19 days), so it is optimal there. In Standard it is 2 days and in Large 5 days above the lower bound; the best possible timetable is not known for these two, because the lower bound need not be reachable.
- The teams finish at nearly the same time: the finish spread of the greedy timetable is 3, 1, 2 and 0 days, against 9, 5, 10 and 0 days for the fixture-order first-fit. The longest wait of a team is also equal or smaller (5, 4, 3, 3 against 6, 4, 7, 3 days).
- The Congested set-up is limited by the rest rule, not by the grounds, so a good first-fit order can also reach the lower bound there; the greedy rule does not add anything in that case.

8.4 Effect of the Selection Rule

To see what the selection rule contributes, the greedy builder was run with three rules (Table 12): the first feasible match in fixture-list order (no priority), the match whose teams have the most matches left, and the rule used in the project, which adds the aging bonus for idle days.

Table 12: Effect of the selection rule

Set-up	Fixture-list order	Matches left only	Matches left + aging (used)
Small	17 days (gap 5, spread 8)	15 days (gap 5, spread 3)	15 days (gap 5, spread 3)
Standard	16 days (gap 4, spread 1)	16 days (gap 4, spread 1)	16 days (gap 4, spread 1)
Large	27 days (gap 9, spread 10)	22 days (gap 3, spread 2)	22 days (gap 3, spread 2)
Congested	19 days (gap 3, spread 0)	19 days (gap 3, spread 0)	19 days (gap 3, spread 0)

The rule “most matches left” is what matters: it saves 2 days in Small and 5 days in Large and it reduces the finish spread from 8 to 3 and from 10 to 2 days, while Standard and Congested are not changed. The aging bonus changed nothing in these four set-ups. Other factors (0, 0.1, 0.5 and 1.0) were also tried: the days stayed the same, and only the factor 1.0 changed the finish spread of Small, from 3 to 2. The aging term is therefore kept only as a small tie-breaker, and its value 0.25 is a design choice that was not tuned further.

8.5 Rounds from the Greedy Colouring

Table 13 shows the greedy colouring of Part 1 for several team numbers. The number of rounds is the lower bound n − 1 only when n is a power of two; otherwise the greedy colouring uses more rounds than the circle method. The rounds describe which matches could be played together, but they do not know about grounds or rest days, so they are not used by the timetable builder.

Table 13: Rounds from greedy colouring vs lower bound

Teams	Matches	Conflicts	Greedy rounds	Lower bound n − 1	Circle method
4	6	12	3	3	3
6	15	60	7	5	5
8	28	168	7	7	7
10	45	360	15	9	9
12	66	660	15	11	11

8.6 Sample Program Output

The following is the output printed by `python cricket_tournament_scheduler.py`. Because the greedy scheduler is deterministic and the random seeds are fixed, the same numbers appear on every run.

```
Self-check of all algorithms: PASSED

PART 1 - greedy colouring of the match-conflict graph
teams matches edges greedy_rounds lower_bound circle_method
    4       6    12             3           3             3
    6      15    60             7           5             5
    8      28   168             7           7             7
   10      45   360            15           9             9
   12      66   660            15          11            11

PART 2 - greedy timetable, Standard set-up (8 teams, 2 grounds, 1 rest day)
  Day  1 | Narendra Modi Stadium: MUM vs CHE | Wankhede Stadium: BLR vs KOL
  Day  2 | Narendra Modi Stadium: DEL vs HYD | Wankhede Stadium: JAI vs AHM
  Day  3 | Narendra Modi Stadium: MUM vs BLR | Wankhede Stadium: CHE vs KOL
  Day  4 | Narendra Modi Stadium: DEL vs JAI | Wankhede Stadium: HYD vs AHM
  Day  5 | Narendra Modi Stadium: MUM vs KOL | Wankhede Stadium: CHE vs BLR
  Day  6 | Narendra Modi Stadium: DEL vs AHM | Wankhede Stadium: HYD vs JAI
  Day  8 | Narendra Modi Stadium: MUM vs DEL | Wankhede Stadium: CHE vs HYD
  Day  9 | Narendra Modi Stadium: BLR vs JAI | Wankhede Stadium: KOL vs AHM
  Day 10 | Narendra Modi Stadium: MUM vs HYD | Wankhede Stadium: CHE vs DEL
  Day 11 | Narendra Modi Stadium: BLR vs AHM | Wankhede Stadium: KOL vs JAI
  Day 13 | Narendra Modi Stadium: MUM vs JAI | Wankhede Stadium: CHE vs AHM
  Day 14 | Narendra Modi Stadium: BLR vs DEL | Wankhede Stadium: KOL vs HYD
  Day 15 | Narendra Modi Stadium: MUM vs AHM | Wankhede Stadium: CHE vs JAI
  Day 16 | Narendra Modi Stadium: BLR vs HYD | Wankhede Stadium: KOL vs DEL

PART 3 - days needed to finish the tournament (random order = mean of 20 seeds)
Scenario   LowerBnd FixtureFF  RandomFF  Greedy  maxgap(F/G)  spread(F/G)
```

Small	15	18	16.6	15	6/5	9/3
Standard	14	18	20.6	16	4/4	5/1
Large	17	27	26.1	22	7/3	10/2
Congested	19	19	22.6	19	3/3	0/0

9. Advantages and Limitations

9.1 Advantages

- Very fast: a complete timetable of a small tournament is built in a few milliseconds.
- Every timetable is valid by construction and is checked by `is_valid()`; the grounds and the rest rule never need manual correction.
- The teams finish at nearly the same time and wait shorter between matches than in the first-fit timetables.
- A lower bound is computed, so the quality of the result can be judged and two set-ups are shown to be optimal.
- Short, readable and deterministic code that uses only the Python standard library.

9.2 Limitations

- The timetable builder is a heuristic; its optimality is not proved and it was 2 and 5 days above the lower bound in the Standard and Large set-ups.
- The greedy colouring needs more rounds than necessary when the number of teams is not a power of two (15 against 9 for 10 teams).
- Only a single round-robin on identical grounds is modelled; home and away matches, travel, weather, television slots, umpires, play-offs and knock-outs are not.
- The comparison uses only two simple first-fit baselines and four set-ups; no exact solver was run, so the distance from the true optimum is unknown for two set-ups.
- The parameters (rest days, aging factor, number of grounds) were chosen by hand.

10. Future Scope

- Compare with an optimal timetable found by integer programming or constraint programming for small tournaments.
- Add a double round-robin with home and away matches and a balance of home matches.
- Add travel distances, venue availability, holidays and television slots as extra rules.
- Add play-off and knock-out stages after the league matches.
- Improve the greedy timetable with local search and tune the parameters automatically.
- Export the timetable to a spreadsheet and add a graphical calendar view.

11. Conclusion

The project applied the greedy method twice to a round-robin cricket tournament: greedy graph colouring to divide the matches into rounds, and a greedy builder to give every match a day and a ground. The colouring gave 7 rounds for 8 teams, which is optimal, but it was wasteful for team numbers that are not a power of two. The timetable builder finished the tournament in 15, 16, 22 and 19 days in the four set-ups, which is 3, 2, 5 and 0 days earlier than the fixture-order first-fit, lower than the random-order first-fit in all of them, and equal to the proven lower bound in two of them. The complexity analysis shows a running time of about $O(M^2)$ for M matches, so the method is suitable for interactive use.

The project also showed the limits of the greedy method: its choices are locally good but not proved globally optimal, the lower bound was not reached in two set-ups, and the results depend on the simplified model. It gave practical experience in modelling a problem as a graph, designing and analysing greedy algorithms, and testing them by experiment.

References

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, *Introduction to Algorithms*, MIT Press.

[2] J. Kleinberg and É. Tardos, *Algorithm Design*, Pearson.

[3] D. J. A. Welsh and M. B. Powell, "An upper bound for the chromatic number of a graph and its application to timetabling problems", *The Computer Journal*, 10(1), 1967, pp. 85–86.

[4] G. Kendall, S. Knust, C. C. Ribeiro and S. Urrutia, "Scheduling in sports: An annotated bibliography", *Computers & Operations Research*, 37(1), 2010, pp. 1–19.

[5] K. Rasmussen and M. A. Trick, "Round robin scheduling – a survey", *European Journal of Operational Research*, 188(3), 2008, pp. 617–636.

[6] Python Software Foundation, Python documentation. https://docs.python.org/3/